

STARTER KIT — 2024 EDITION

Python Automation & Tech **Starter Kit**

Your complete beginner-friendly guide to automating everyday tasks with Python. Copy-paste scripts included.

By **Ali Raza Saleem** • TechKit

Table of Contents

Getting Started with Python	Chapter 01
Auto-Organize Your Files	Chapter 02
Bulk Rename Files Like a Pro	Chapter 03
Web Scraping — Extract Data from Websites	Chapter 04
Automate WhatsApp Messages	Chapter 05
Email Automation with Python	Chapter 06
PDF Merger & Splitter Tool	Chapter 07
Password Generator Script	Chapter 08
System Info & Health Monitor	Chapter 09
Next Steps & Resources	Chapter 10

CHAPTER 01

Getting Started with Python

Install Python, set up your environment, and run your first script.

Step 1: Install Python

Download the latest version of Python from the official website. Make sure to check the box that says **"Add Python to PATH"** during installation.

1. Go to python.org/downloads
2. Download Python 3.12+ for your OS (Windows/Mac/Linux)
3. Run the installer — **check "Add Python to PATH"**
4. Click Install Now

Step 2: Verify Installation

Open your terminal (Command Prompt on Windows, Terminal on Mac/Linux) and run:

TERMINAL

```
python --version
```

OUTPUT

```
Python 3.12.4
```

💡 **Tip:** If you see a version number, you're all set! If not, restart your computer and try again.

Step 3: Install a Code Editor

We recommend **Visual Studio Code (VS Code)** — it's free, lightweight, and perfect for Python.

1. Download from code.visualstudio.com
2. Install the **Python extension** from the Extensions tab
3. Create a new file called `hello.py`

Your First Script

HELLO.PY

```
print("Hello, World! 🚀")  
print("Welcome to the Python Automation Starter Kit!")  
print("Let's build something amazing together.")
```

Run it from the terminal:

TERMINAL

```
python hello.py
```

OUTPUT

```
Hello, World! 🚀 Welcome to the Python Automation Starter Kit! Let's build  
something amazing together.
```



 **Info:** You just ran your first Python script! Every chapter from here will build on this foundation.

CHAPTER 02

Auto-Organize Your Files

Automatically sort messy downloads folders by file type.

Tired of a cluttered Downloads folder? This script automatically moves files into folders based on their type — images go to `Images/`, PDFs go to `Documents/`, and so on.

How It Works

- Scans every file in the target folder
- Identifies the file extension (.jpg, .pdf, .mp4, etc.)
- Creates category folders if they don't exist
- Moves each file to the correct folder automatically

The Script

FILE_ORGANIZER.PY

```
import os
import shutil

# Change this to your target folder
TARGET_DIR = os.path.expanduser("~/Downloads")

# Define file categories and their extensions
CATEGORIES = {
    "Images":    [".jpg", ".jpeg", ".png", ".gif", ".svg", ".webp"],
```

```
"Documents": [".pdf", ".doc", ".docx", ".txt", ".xlsx", ".pptx"],
"Videos":    [".mp4", ".mkv", ".avi", ".mov"],
"Audio":     [".mp3", ".wav", ".flac", ".aac"],
"Archives":  [".zip", ".rar", ".7z", ".tar", ".gz"],
"Code":      [".py", ".js", ".html", ".css", ".json"],
}
```

```
def organize_files():
    moved_count = 0

    for filename in os.listdir(TARGET_DIR):
        filepath = os.path.join(TARGET_DIR, filename)

        # Skip directories
        if os.path.isdir(filepath):
            continue

        # Get file extension
        _, ext = os.path.splitext(filename)
        ext = ext.lower()

        # Find the right category
        destination_folder = "Others"
        for category, extensions in CATEGORIES.items():
            if ext in extensions:
                destination_folder = category
                break

        # Create folder if needed & move file
        dest_path = os.path.join(TARGET_DIR, destination_folder)
        os.makedirs(dest_path, exist_ok=True)
        shutil.move(filepath, os.path.join(dest_path, filename))
        moved_count += 1
        print(f"  {filename} → {destination_folder}/")

    print(f"\n🎉 Done! Organized {moved_count} files.")

if __name__ == "__main__":
```

```
print("📁 Organizing your files...\n")  
organize_files()
```

SAMPLE OUTPUT

```
📁 Organizing your files... ✅ photo_2024.jpg → Images/ ✅ assignment.pdf →  
Documents/ ✅ lecture.mp4 → Videos/ ✅ song.mp3 → Audio/ ✅ project.zip →  
Archives/ 🎉 Done! Organized 5 files.
```

💡 **Tip:** You can customize the CATEGORIES dictionary to add your own file types and folder names. Add .apk to a new "Apps" category, for example!

CHAPTER 03

Bulk Rename Files Like a Pro

Rename hundreds of files in seconds with a pattern.

Need to rename 200 photos from `IMG_3847.jpg` to `vacation_001.jpg`? This script does it in one click.

The Script

BULK_RENAME.PY

```
import os

# Configuration
FOLDER = os.path.expanduser("~/Pictures/Rename")
PREFIX = "photo"          # New name prefix
START_NUM = 1             # Start counting from
PADDING = 3               # Number padding (001, 002, 003...)

def bulk_rename():
    files = sorted(os.listdir(FOLDER))
    count = 0

    for i, filename in enumerate(files, start=START_NUM):
        filepath = os.path.join(FOLDER, filename)

        if os.path.isdir(filepath):
            continue
```

```
_, ext = os.path.splitext(filename)
new_name = f"{PREFIX}_{str(i).zfill(PADDING)}{ext}"
new_path = os.path.join(FOLDER, new_name)

os.rename(filepath, new_path)
print(f" 📄 {filename} → {new_name}")
count += 1

print(f"\n🎉 Renamed {count} files!")

if __name__ == "__main__":
    print("✎ Bulk Rename Tool\n")
    bulk_rename()
```

SAMPLE OUTPUT

```
✎ Bulk Rename Tool 📄 IMG_3847.jpg → photo_001.jpg 📄 IMG_3848.jpg →
photo_002.jpg 📄 IMG_3849.jpg → photo_003.jpg 🎉 Renamed 3 files!
```

⚠️ **Note:** Always test on a small folder first. Renaming is hard to undo — keep backups!

CHAPTER 04

Web Scrapping — Extract Data from Websites

Pull headlines, prices, or any data from websites automatically.

Install Required Library

TERMINAL

```
pip install requests beautifulsoup4
```

The Script — Scrape News Headlines

SCRAPER.PY

```
import requests
from bs4 import BeautifulSoup

URL = "https://news.ycombinator.com/"

def scrape_headlines():
    print("🌐 Fetching headlines...\n")

    response = requests.get(URL)
    soup = BeautifulSoup(response.text, "html.parser")
```

```
# Find all headline links
headlines = soup.select(".titleline > a")

for i, headline in enumerate(headlines[:10], 1):
    title = headline.text
    link = headline.get("href", "#")
    print(f" {i}. {title}")
    print(f"     📄 {link}\n")

print(f"✅ Scraped {min(10, len(headlines))} headlines!")

if __name__ == "__main__":
    scrape_headlines()
```

SAMPLE OUTPUT

```
🌐 Fetching headlines... 1. Show HN: I built an open-source AI tool 📄
https://example.com/article1 2. Python 3.13 new features explained 📄
https://example.com/article2 ✅ Scraped 10 headlines!
```

💡 **Tip:** You can modify the `soup.select()` line to target any HTML element on any website. Right-click → Inspect Element to find the CSS selector!

CHAPTER 05

Automate WhatsApp Messages

Schedule and send WhatsApp messages automatically using Python.

Install Required Library

TERMINAL

```
pip install pywhatkit
```

The Script

WHATSAPP_BOT.PY

```
import pywhatkit as kit

# Send a message to a phone number at a specific time
# Format: +country_code + number
PHONE_NUMBER = "+923001234567"
MESSAGE = "Hey! This message was sent automatically using Python 🐍🚀"

# Send at 2:30 PM (24-hour format)
HOUR = 14
MINUTE = 30

def send_message():
    print(f"📱 Scheduling WhatsApp message...")
```

```
print(f"    To: {PHONE_NUMBER}")
print(f"    At: {HOUR}:{MINUTE:02d}")
print(f"    Msg: {MESSAGE}\n")

kit.sendwhatmsg(PHONE_NUMBER, MESSAGE, HOUR, MINUTE)
print("✅ Message sent successfully!")

if __name__ == "__main__":
    send_message()
```

⚠️ **Note:** WhatsApp Web must be logged in on your browser. The script opens a browser tab to send the message. Keep the browser open until the message is sent.

Bonus: Send to a Group

WHATSAPP_GROUP.PY

```
# Send message to a WhatsApp group
# Use the group invite link ID
GROUP_ID = "YourGroupLinkID"

kit.sendwhatmsg_to_group(GROUP_ID, "Hello group! 🤖", 15, 0)
```

CHAPTER 06

Email Automation with Python

Send professional emails programmatically — perfect for newsletters and notifications.

The Script

EMAIL_SENDER.PY

```
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

# Your Gmail credentials
SENDER_EMAIL = "your.email@gmail.com"
APP_PASSWORD = "your-app-password" # NOT your Gmail password

# Recipient
TO_EMAIL = "recipient@example.com"

def send_email():
    # Create message
    msg = MIMEMultipart()
    msg["From"] = SENDER_EMAIL
    msg["To"] = TO_EMAIL
    msg["Subject"] = "Hello from Python! 🐍"

    body = """
    Hi there!

    This email was sent automatically using Python.
```

```
Pretty cool, right?

Best regards,
Your Python Script 🤖
"""

msg.attach(MIMEText(body, "plain"))

# Send email
print("📧 Sending email...")
with smtplib.SMTP("smtp.gmail.com", 587) as server:
    server.starttls()
    server.login(SENDER_EMAIL, APP_PASSWORD)
    server.sendmail(SENDER_EMAIL, TO_EMAIL, msg.as_string())

print("✅ Email sent successfully!")

if __name__ == "__main__":
    send_email()
```

i Info: For Gmail, you need to generate an **App Password**. Go to Google Account → Security → 2-Step Verification → App Passwords → Generate.

CHAPTER 07

PDF Merger & Splitter Tool

Combine multiple PDFs into one or extract specific pages.

Install Required Library

TERMINAL

```
pip install PyPDF2
```

Merge Multiple PDFs

PDF_MERGER.PY

```
from PyPDF2 import PdfMerger

def merge_pdfs(file_list, output_name="merged.pdf"):
    merger = PdfMerger()

    for pdf_file in file_list:
        print(f" 📄 Adding: {pdf_file}")
        merger.append(pdf_file)

    merger.write(output_name)
    merger.close()
    print(f"\n✅ Merged into: {output_name}")
```

```
# List your PDF files here
files_to_merge = [
    "chapter1.pdf",
    "chapter2.pdf",
    "chapter3.pdf",
]

if __name__ == "__main__":
    print("📄 PDF Merger Tool\n")
    merge_pdfs(files_to_merge)
```

Extract Specific Pages

PDF_SPLITTER.PY

```
from PyPDF2 import PdfReader, PdfWriter

def extract_pages(input_pdf, pages, output_name="extracted.pdf"):
    reader = PdfReader(input_pdf)
    writer = PdfWriter()

    for page_num in pages:
        writer.add_page(reader.pages[page_num - 1])
        print(f"📄 Extracted page {page_num}")

    with open(output_name, "wb") as f:
        writer.write(f)

    print(f"\n✅ Saved to: {output_name}")

# Extract pages 1, 3, and 5 from a PDF
if __name__ == "__main__":
    print("🔪 PDF Splitter Tool\n")
    extract_pages("document.pdf", pages=[1, 3, 5])
```

CHAPTER 08

Password Generator Script

Generate strong, random passwords for all your accounts.

The Script

PASSWORD_GEN.PY

```
import random
import string

def generate_password(length=16, use_special=True):
    chars = string.ascii_letters + string.digits
    if use_special:
        chars += string.punctuation

    # Ensure at least one of each type
    password = [
        random.choice(string.ascii_uppercase),
        random.choice(string.ascii_lowercase),
        random.choice(string.digits),
    ]
    if use_special:
        password.append(random.choice(string.punctuation))

    # Fill remaining length
    remaining = length - len(password)
    password.extend(random.choices(chars, k=remaining))

    # Shuffle to randomize position
```

```
    random.shuffle(password)
    return "".join(password)

if __name__ == "__main__":
    print("🔒 Password Generator\n")
    print("    Generated passwords:\n")

    for i in range(5):
        pwd = generate_password(length=16)
        print(f"    {i+1}. {pwd}")

    print("\n    ✅ Use a password manager to store them safely!")
```

SAMPLE OUTPUT

```
🔒 Password Generator Generated passwords: 1. K#9mX$2pLqR&8wNf 2. Zt!5vBn@3xYcJ7sE
3. P8rW*4hDm!6aQf2U 4. Gx#7LkS9$NvRt5mC 5. Fj3!Yz8W@pQb6XhN ✅ Use a password
manager to store them safely!
```

CHAPTER 09

System Info & Health Monitor

Check CPU, memory, disk usage, and system health in real-time.

Install Required Library

TERMINAL

```
pip install psutil
```

The Script

SYSTEM_MONITOR.PY

```
import psutil
import platform
from datetime import datetime

def system_report():
    print("=" * 50)
    print("      🖥️ SYSTEM HEALTH REPORT")
    print("=" * 50)

    # OS Info
    print(f"\n💻 OS: {platform.system()} {platform.release()}")
    print(f"🏠 Machine: {platform.machine()}")
    print(f"🕒 Time: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
```

```

# CPU
cpu_percent = psutil.cpu_percent(interval=1)
cpu_count = psutil.cpu_count()
print(f"\n⚡ CPU Usage: {cpu_percent}%")
print(f"    Cores: {cpu_count}")

# Memory
mem = psutil.virtual_memory()
print(f"\n🧠 RAM: {mem.used / (1024**3):.1f} GB / {mem.total / (1024**3):.1f} GB")
print(f"    Usage: {mem.percent}%")

# Disk
disk = psutil.disk_usage("/")
print(f"\n💾 Disk: {disk.used / (1024**3):.1f} GB / {disk.total / (1024**3):.1f} GB")
print(f"    Usage: {disk.percent}%")

# Battery (if laptop)
battery = psutil.sensors_battery()
if battery:
    status = "⚡ Charging" if battery.power_plugged else "🔋 On Battery"
    print(f"\n🔋 Battery: {battery.percent}% ({status})")

print("\n" + "=" * 50)
print("    ✅ System is healthy!" if cpu_percent < 80 else "    ⚠️ CPU usage is high")
print("\n" + "=" * 50)

if __name__ == "__main__":
    system_report()

```

SAMPLE OUTPUT

```

===== 🖥️ SYSTEM HEALTH REPORT
===== 🖥️ OS: Windows 11 🚫 Machine:
AMD64 🕒 Time: 2024-09-11 15:30:00 ⚡ CPU Usage: 23% Cores: 8 🧠 RAM: 6.2 GB /
16.0 GB Usage: 38.7% 💾 Disk: 145.3 GB / 476.9 GB Usage: 30.5% 🔋 Battery: 78% (⚡)

```

```
Charging) =====  
=====
```



System is healthy!

CHAPTER 10

Next Steps & Resources

Where to go from here and how to level up your Python skills.



Congratulations!

You've just completed the **Python Automation & Tech Starter Kit**. You've learned how to automate file management, web scraping, messaging, emails, PDFs, and system monitoring. These are real-world skills that save hours of manual work.

What to Learn Next

- **APIs & Automation:** Learn to use REST APIs to connect apps (Telegram bots, Twitter automation, weather dashboards)
- **Task Scheduling:** Use `schedule` or `cron` to run your scripts automatically at specific times
- **GUI Apps:** Build desktop apps with `tkinter` or `PyQt`
- **Web Development:** Build websites with `Flask` or `Django`
- **Data Science:** Explore `pandas`, `matplotlib`, and `jupyter notebooks`

Free Resources

- **Python Docs:** docs.python.org
- **Real Python:** realpython.com — In-depth tutorials
- **Automate the Boring Stuff:** automatetheboringstuff.com — Free online book
- **YouTube:** Search "Python automation projects" for video walkthroughs

Pro Tips for Your Journey

💡 **Tip:** Build projects, not tutorials. The best way to learn is by solving YOUR problems with code.

💡 **Tip:** Use GitHub to store your scripts. It's your portfolio and backup in one place.

💡 **Tip:** Join communities — r/learnpython, Python Discord, and local meetups. Learning alone is hard; learning together is fun.

Thank You! 🙏

Thank you for purchasing this kit. Your support means the world and helps me create more content for the tech community. If you have any questions, feedback, or script requests — reach out anytime!

Python Automation & Tech Starter Kit

© 2024 Ali Raza Saleem • TechKit

All scripts are provided as-is for educational purposes.

Unauthorized redistribution is prohibited.